GUÍA DE ARQUITECTURA & ARRANQUE

Sistema de Información Hospitalario

Cómo estructurar el proyecto en Vue 3 + NestJS + PostgreSQL aplicando Clean Architecture y SOLID, antes de escribir la primera línea de código de negocio.

Vue 3 | NestJS | TypeORM | PostgreSQL | Clean Architecture | SOLID

01

Qué pide realmente el proyecto

Con base en *Diseño de sistema* e *Historias de usuario*, el sistema tiene 6 módulos: uno transversal (Identidad y Accesos) y cinco de negocio (Admisión y Turnos, Historia Clínica Electrónica, Atención y Enfermería, Recursos y Camas, Facturación).

Lo que cambia el punto de partida es que las historias de usuario no describen CRUDs simples. Piden explícitamente resolver **condiciones de carrera** (dos recepcionistas reservando el mismo turno, dos administrativos ocupando la misma cama en el mismo milisegundo), una **máquina de estados** para turnos y camas, un **Builder** para ensamblar protocolos de cuidado según el motivo de ingreso, y un **Strategy** para calcular la facturación según el tipo de cobertura. La arquitectura no es un detalle de implementación acá: es el contenido pedagógico del proyecto. Por eso este informe prioriza decisiones de arquitectura y orden de trabajo por sobre pantallas o features.

MÓDULO	ROL EN EL SISTEMA	LO QUE EXIGE DE ARQUITECTURA
IAM	Transversal: autenticación, roles (Admin, Médico, Enfermero, Recepcionista, Paciente)	Guards + RBAC reutilizables en todos los módulos
Admisión y Turnos	Agenda médica, reservas, entreturnos, lista de espera, triaje	Bloqueo de concurrencia, State, cola con prioridad
Historia Clínica (HCE)	Ficha del paciente, alergias, antecedentes, evoluciones	Modelo de datos rico, integridad referencial
Atención y Enfermería	Signos vitales, medicación, protocolos de cuidado	Builder, alertas por umbral, descuento de stock
Recursos y Camas	Mapa de camas, inventario de insumos	Bloqueo de concurrencia, State, eventos

MÓDULO	ROL EN EL SISTEMA	LO QUE EXIGE DE ARQUITECTURA
Facturación	Cálculo de cargos según cobertura	Strategy, integración asíncrona por eventos

02

Arquitectura en capas

La forma más directa de aplicar SOLID de punta a punta es organizar el backend NestJS en capas al estilo Clean Architecture, donde la **regla de dependencia** es la que hace el trabajo pesado: cada capa solo puede conocer a la capa inmediatamente interior, y el Dominio no conoce nada de NestJS, TypeORM ni PostgreSQL.



Cada capa llama hacia adentro; Infraestructura no es llamada por Dominio, sino que implementa las interfaces que Dominio define — así TypeORM y PostgreSQL son un detalle reemplazable.

En la práctica dentro de NestJS: `Domain` es TypeScript sin dependencias de Nest, con las entidades, los value objects y las *interfaces* de repositorio (`ITurnoRepository`). `Application` son los casos de uso, inyectados vía `@Injectable()`, que dependen de esas interfaces. `Infrastructure` implementa esas interfaces con `@InjectRepository`

de TypeORM y se registra en el módulo de Nest con un token (`{ provide: TURNO_REPOSITORY, useClass: TypeOrmTurnoRepository }`). Eso es Dependency Inversion aplicado, no solo declarado.

> **CORRECCIÓN** En el código real del proyecto las entidades de dominio llevan los decoradores de TypeORM (`@Entity`, `@Column`) para no mantener dos clases por concepto. Es una desviación consciente de Clean Architecture estricta: lo que sí se respeta es que los casos de uso dependen de las *interfaces* de repositorio y nunca del repositorio de TypeORM ni de `EntityManager`.

03

SOLID, aplicado a este dominio

Los cinco principios en abstracto no ayudan a arrancar; lo que ayuda es saber dónde va a doler cada uno en *este* proyecto puntual.

PRINCIPIO	DÓNDE SE APLICA ACÁ
SRP	Un caso de uso por acción de negocio: `ReservarTurnoUseCase`, `AsignarCamaUseCase`, `CalcularCargoUseCase` — no un `TurnoService` con veinte métodos que mezcla reservar, cancelar y reportar.
OCP	El Strategy de facturación: agregar una cobertura nueva (ART, por ejemplo) es una clase nueva que implementa `IEstrategiaCobertura`, no un `if` más en un método que ya calcula para pública, prepaga y particular.
LSP	Cualquier implementación de `ITurnoRepository` o `ICamaRepository` — la real con TypeORM, o una en memoria para tests — tiene que poder sustituir a la otra sin que el caso de uso lo note.
ISP	DTOs específicos por operación (`CrearTurnoDto`, `CancelarTurnoDto`) en vez de un `TurnoDto` gigante con la mitad de los campos opcionales según la acción.
DIP	Los casos de uso dependen de interfaces definidas en Dominio, nunca de TypeORM directamente. Es lo que hace testeable el Patrón State y el Strategy sin levantar una base de datos.

04

Patrones de diseño que pide cada épica

Esta tabla es, en esencia, la traducción técnica de *Historias de usuario*: qué patrón resuelve cada requisito y en qué capa vive.

MÓDULO · ÉPICA	PATRÓN / TÉCNICA	POR QUÉ
Admisión — Reserva de turnos (HU 1.1)	`Índice único + locking optimista`	Dos recepcionistas reservan el mismo turno: el índice único parcial deja pasar solo un INSERT, y las modificaciones posteriores usan un UPDATE condicional por versión.
Admisión — Máquina de estados (Épica 2)	`State`	Transiciones válidas de Turno (Programado → Confirmado → En Sala de Espera → Atendido, o Cancelado) no pueden vivir en un `switch` disperso.
Admisión — Triaje (Épica 3)	`Cola de prioridad`	Código Rojo debe insertarse al frente de la cola de espera, no al final.
Atención — Protocolos de cuidado (Épica 2)	`Builder`	El plan de cuidado inicial cambia según el motivo de ingreso (Pre-Op, Respiratorio); un Builder ensambla la secuencia de tareas paso a paso.
Camas — Asignación (Épica 1)	`UPDATE condicional`	Misma condición de carrera que en turnos, aplicada a la cama.
Camas — Máquina de estados (Épica 2)	`State`	Libre → Ocupada → En limpieza → Libre; una cama nunca salta de Ocupada a Libre sin pasar por limpieza.
Facturación — Cálculo de costos (Épica 1)	`Strategy`	El cálculo cambia según cobertura (pública, prepaga, particular) sin tocar código existente al agregar una nueva.
Facturación — Integración async (Épica 2)	`Observer / eventos de dominio`	Back-Office reacciona a `MedicacionAdministrada` sin bloquear la operación de enfermería.

Tablas de estado, tal como las definiste

ESTADO DEL TURNO	MOMENTO DE APLICACIÓN
Programado	Al finalizar la reserva inicial
Confirmado	Cuando el paciente revalida su intención de asistir
En Sala de Espera	Al registrarse físicamente en recepción
Atendido	Al concluir la consulta
Cancelado	Si el paciente o la clínica anulan la cita

ESTADO DE LA CAMA	TRANSICIÓN PERMITIDA
Libre	Puede asignarse a un paciente ingresante
Ocupada	Solo puede pasar a "en limpieza" tras alta o traslado
En limpieza	Bloqueada hasta que maestranza confirme higiene
Bloqueada	Fuera de servicio por mantenimiento de infraestructura

05

Modelo de datos inicial

No hace falta el modelo completo para arrancar, pero sí las entidades núcleo por módulo — son las que van a definir las primeras migraciones de TypeORM.

MÓDULO	ENTIDADES CLAVE
IAM	`Usuario, Rol, Permiso`
Admisión	`Paciente, Medico, Turno, ColaEntreturno`
HCE	`HistoriaClinica, Alergia, AntecedenteFamiliar, Evolucion`
Atención	`SignoVital, IndicacionMedica, ProtocoloCuidado, TareaProtocolo`
Camas	`Cama, AsignacionCama, Insumo, StockInsumo`
Facturación	`Cobertura, Factura, LineaFactura, EventoDominio (outbox)`

NOTA `Turno` y `Cama` son las dos tablas que necesitan una columna de control de concurrencia desde el primer día — ver sección siguiente. `EventoDominio` es opcional pero recomendado: guardar el evento `MedicacionAdministrada` como fila antes de publicarlo simplifica la trazabilidad que pide la historia de Facturación (Épica 2).

06

Concurrencia en PostgreSQL

Las historias 1.1 (turnos) y 1.1 de Camas piden lo mismo con otro nombre: que la segunda petición simultánea pierda con un error claro, no con datos corruptos. Hay dos situaciones distintas y cada una se resuelve con una herramienta diferente, ambas portables entre motores y sin depender de un tipo especial de la base (eso era lo que hacía `rowversion` en SQL Server).

CORRECCIÓN La versión anterior de esta sección decía que `@VersionColumn()` compara la versión en el `WHERE` de cada `UPDATE` y que `save()` lanza `OptimisticLockVersionMismatchError`. Eso es incorrecto: `@VersionColumn()` solo **incrementa** el contador en cada actualización, y `save()` no compara nada, así que dos escrituras simultáneas se pisan sin error. La comparación hay que escribirla a mano, como se muestra abajo.

Caso 1: dos personas crean el mismo turno (INSERT)

Acá no hay una fila previa que versionar: las dos peticiones insertan. Lo que garantiza que solo una gane es un **índice único parcial** que la base hace cumplir, sin importar el orden de llegada. El que pierde recibe el error 23505 de PostgreSQL y el caso de uso lo traduce a 409.

```
// entidad TypeORM
@Entity('turnos')
@Index('uq_turno_medico_horario', ['medicoId', 'fechaHora'], {
  unique: true,
  where: "estado <> 'cancelado'",   // un turno cancelado libera el horario
})
export class Turno {
  @PrimaryGeneratedColumn('uuid') id: string;
  @Column('uuid') medicoId: string;
  @Column('timestamptz') fechaHora: Date;
  @Column({ default: 'programado' }) estado: EstadoTurno;
  @VersionColumn() version: number;   // lo usa el caso 2
}

// shared/database/errores-pg.ts
export function esViolacionDeUnicidad(e: unknown): boolean {
  return (e as { code?: string })?.code === '23505'
      || (e as { driverError?: { code?: string } })?.driverError?.code === '23505';
}

// en el caso de uso
try {
  return await this.turnos.guardar(turno);
} catch (e) {
  if (esViolacionDeUnicidad(e)) {
    throw new ConflictoDeConcurrenciaException('Ese horario acaba de ser tomado');
  }
  throw e;
}
```

Caso 2: dos personas modifican la misma fila (UPDATE)

Confirmar un turno, asignar una cama, cambiar el estado de una factura: acá sí hay una fila leída con una versión. La forma correcta es un UPDATE condicional que incluye la versión leída en el WHERE y mira cuántas filas afectó. Si afectó cero, otra persona se adelantó.

```
// infraestructura: repositorio TypeORM
async actualizarEstado(turno: Turno, nuevoEstado: EstadoTurno): Promise<void> {
  const r = await this.repo
    .createQueryBuilder()
    .update(Turno)
    .set({ estado: nuevoEstado, version: () => 'version + 1' })
    .where('id = :id AND version = :versionLeida', {
      id: turno.id,
      versionLeida: turno.version,   // la que tenía la fila cuando se leyó
    })
    .execute();

  if (r.affected !== 1) {
    throw new ConflictoDeConcurrenciaException('El turno fue modificado por otra persona');
  }
}
```

Para "tomar" algo que debe estar libre (una cama, un stock), el mismo principio se resuelve validando el estado previo dentro del UPDATE, sin necesidad de leer antes:

```
UPDATE camas
SET estado = 'ocupada', version = version + 1
WHERE id = $1 AND estado = 'libre';
-- 0 filas afectadas → otra persona la ocupó primero → se devuelve 409

UPDATE stock_insumos
SET cantidad = cantidad - $2
WHERE insumo_id = $1 AND cantidad >= $2;
-- 0 filas afectadas → no alcanza el stock → se devuelve 422
```

CUÁNDO USAR PESIMISTA EN VEZ

Si en algún punto necesitás bloquear una fila mientras se ejecuta lógica adicional (por ejemplo, validar cobertura antes de confirmar una cama), usá `SELECT ... FOR UPDATE` dentro de una transacción explícita — es el equivalente en PostgreSQL del `WITH (UPDLOCK, ROWLOCK)` de T-SQL. Para casi todos los casos de este proyecto alcanzan el índice único (para crear) y el UPDATE condicional (para modificar), y son más simples de explicar y de testear.

El código completo de estos patrones, aplicado a cada módulo, está en el *Informe de Módulos HIS*.

07

Estructura de carpetas

Un monolito modular — un módulo de Nest por módulo de negocio, cada uno con sus cuatro capas adentro — evita tanto el "todo en un service gigante" como la sobre-ingeniería de microservicios que este alcance no necesita.

Backend (NestJS)

```
src/
├── shared/                    # kernel: excepciones, decoradores, guards RBAC
├── iam/
│   ├── domain/                # Usuario, Rol (entidades de dominio)
│   ├── application/           # LoginUseCase, guards, estrategias JWT
│   ├── infrastructure/        # TypeORM, Passport
│   └── presentation/          # AuthController
├── admisiones/
│   ├── domain/                # Turno, EstadoTurno (State), ITurnoRepository
│   ├── application/           # ReservarTurnoUseCase, InsertarEntreturnoUseCase
│   ├── infrastructure/        # TypeOrmTurnoRepository
│   └── presentation/          # TurnosController, DTOs
├── historia-clinica/
├── atencion-enfermeria/       # incluye los Protocol Builders
├── camas/                     # reutiliza el State del mismo modo que admisiones
├── facturacion/                # Strategy de cobertura + listener de eventos
└── main.ts
```

Frontend (Vue 3 + Vite + TS)

```
src/
├── shared/                    # componentes UI, cliente http (axios), tipos
├── modules/
│   ├── admisiones/
│   │   ├── views/             # AgendaView.vue, TriajeView.vue
│   │   ├── components/
│   │   ├── stores/            # Pinia: useTurnosStore
│   │   └── api/                # turnos.api.ts
│   ├── camas/
│   ├── atencion/
│   └── facturacion/
├── router/                     # rutas + guards por rol (RBAC en UI)
└── main.ts
```

08

Librerías que resuelven cada necesidad

NECESIDAD	ELECCIÓN RECOMENDADA	NOTA
ORM para PostgreSQL	`TypeORM`	Soporta pg (node-postgres) nativo y migraciones. La concurrencia se resuelve con índices únicos y UPDATE condicionales vía QueryBuilder (ver sección 06).
Cliente de base de datos	`DataGrip`	Gratis para uso no comercial (JetBrains, desde oct. 2025). Autocompletado, browser de esquema y consultas contra el Postgres local.
Autenticación	`@nestjs/passport + passport-jwt`	Guards por rol reutilizables en todos los controllers.
Validación de DTOs	`class-validator + class-transformer`	Se integra directo con `ValidationPipe` global de Nest.
Documentación de API	`@nestjs/swagger`	Útil para que el frontend Vue avance en paralelo sin esperar al backend terminado.
Testing backend	`Jest + Supertest`	Los casos de uso, al depender de interfaces, se testean con repositorios en memoria — sin PostgreSQL real.
Estado en frontend	`Pinia`	Un store por módulo, alineado a la misma división del backend.
Formularios	`VeeValidate + Zod`	Mismo esquema de validación que el backend, sin duplicar reglas de negocio.
Eventos de dominio	`EventEmitter2 (nativo de Nest)`	Suficiente para simular el desacople async de Facturación sin sumar un broker externo.

09

Plan de arranque por fases

El orden importa: conviene validar la arquitectura completa (capas + concurrencia + un patrón) en un solo módulo antes de repetir el molde en los demás. Admisión y Turnos concentra la mayor densidad de requisitos técnicos (concurrencia + State + cola), así que es el mejor candidato para ir primero — lo aprendido ahí se reutiliza directo en Camas.

0

Entorno y scaffolding

Repositorio, Nest CLI, Vite + Vue 3 + TS, PostgreSQL (nativo o Docker, a elección de cada uno — sección 12), conexión TypeORM, ESLint + Prettier, estructura de carpetas de la sección 07.

`sin lógica de negocio aún`

1

Núcleo transversal: IAM

Login, JWT, roles y guards RBAC. Filtro global de excepciones y logging. Todo lo demás depende de esto.

2

Admisión y Turnos (slice vertical completo)

CRUD base de Paciente/Médico/Turno → concurrencia optimista (HU 1.1) → State para el ciclo de vida del turno (Épica 2) → cola de triaje (Épica 3) → entreturnos y lista de espera.

`locking optimista` `State` `cola de prioridad`

3

Recursos y Camas

Reutiliza el mismo enfoque de concurrencia y State ya probado en Turnos — debería ser notablemente más rápido de construir.

`reutiliza patrón de fase 2`

4

HCE + Atención y Enfermería

Historia clínica base y signos vitales (mayormente CRUD con alertas por umbral) junto con los Protocol Builders (Pre-Op, Respiratorio) y el descuento de inventario al administrar medicación.

`Builder`

5

Facturación

Strategy por tipo de cobertura, y el listener de eventos que escucha `MedicacionAdministrada` para generar cargos automáticos. Se beneficia de que los demás módulos ya emitan datos reales.

`Strategy` `eventos`

6

Frontend integral + pulido

Pantallas Vue conectadas a cada módulo, RBAC reflejado en la UI, tests end-to-end de los flujos de concurrencia (el caso más valioso para demostrar), documentación final.

10

Primeros comandos, hoy

El PostgreSQL local se levanta de dos formas — instalación nativa en Windows o el `docker-compose.yml` del repo — el detalle de las dos está en la sección 12. Cualquiera de las dos deja el motor escuchando en `localhost:5432`; a partir de ahí los comandos son los mismos:

```
# Backend
npm i -g @nestjs/cli
nest new backend --package-manager npm
cd backend
npm i @nestjs/typeorm typeorm pg
npm i @nestjs/passport passport passport-jwt @nestjs/jwt
npm i class-validator class-transformer
npm i @nestjs/swagger
npm i -D jest supertest
cd ..

# Frontend
npm create vite@latest frontend -- --template vue-ts
cd frontend
npm i pinia vue-router axios
npm i -D vee-validate zod
```

11

Checklist de Clean Code desde el día uno

- Ningún caso de uso (carpeta `application/`) importa `typeorm` ni inyecta un `Repository` de TypeORM: dependen solo de las interfaces del dominio. Las entidades de `domain/` sí llevan decoradores de TypeORM.
- Cada caso de uso recibe sus dependencias por constructor (inyección de Nest), nunca las instancia con `new`.
- Los DTOs de entrada y las entidades de dominio son clases distintas — nunca se expone la entidad directamente en un controller.
- Toda transición de estado (Turno, Cama) pasa por el método del patrón State, nunca por una asignación directa al campo `estado`.
- Los tests de casos de uso corren sin base de datos real, usando implementaciones en memoria de los repositorios.
- Cada módulo de Nest expone solo lo que otros módulos necesitan importar — nada de acceder al repositorio de otro módulo directamente.

Trabajo en equipo

Con varios estudiantes tocando el mismo modelo de datos, conviene separar dos cosas: el motor que cada uno usa para desarrollar día a día, y el lugar donde se prueba la integración. Con lo que este proyecto pide — bloqueos de concurrencia, máquina de estados de turnos y camas — una única base compartida entre todos se rompe rápido: alguien trunca una tabla para probar su historia, otro queda a mitad de una transacción, y los tests de concurrencia de uno interfieren con los del resto.

SE COMPARTE (EN EL REPO)	NO SE COMPARTE (POR ESTUDIANTE)
Versión de PostgreSQL acordada (17) y nombre de la base (`HospitalHIS`)	La instancia de PostgreSQL en sí — cada uno levanta la suya, nativa o con Docker
Migraciones de TypeORM commiteadas	El estado de los datos en un momento dado
Script de seed (roles, coberturas, camas de ejemplo)	`.env` con la password real

Cómo levanta cada uno su PostgreSQL: nativo o Docker

A diferencia de SQL Server, acá da lo mismo cuál elija cada estudiante — lo único que importa es que todos corran **PostgreSQL 17** y la base se llame `HospitalHIS`, porque eso es lo que garantiza que las migraciones commiteadas se apliquen igual en cualquier máquina. El motor en sí nunca se comparte (ver tabla arriba): cada uno levanta el suyo.

Instalación nativa — recomendada si no querés depender de Docker Desktop. PostgreSQL corre nativo en Windows, Mac y Linux por igual, y a diferencia de SQL Server el instalador no pide configurar un modo de autenticación especial ni habilitar TCP/IP a mano: baja de postgresql.org, se instala como servicio, y queda escuchando en el puerto 5432 con la contraseña que le pongas al usuario `postgres` durante la instalación (esa es la que va en tu `.env`). Un solo paso manual que el contenedor Docker no necesita: crear la base la primera vez — con DataGrip, clic derecho sobre el server → *New* → *Database* → nombre `HospitalHIS`.

Docker — para quien prefiera no instalar nada en el sistema. El `docker-compose.yml` de abajo sigue en el repo para quien lo quiera usar; con `POSTGRES_DB` ya definido, crea la base sola en el primer `docker compose up -d`, sin el paso manual que sí pide la instalación nativa.

```
services:
  postgres:
    image: postgres:17
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: "${DB_PASSWORD}"
      POSTGRES_DB: HospitalHIS
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

Con un `.env.example` commiteado (sin la password real) y `.env` en el `.gitignore`, cada uno completa su propia password local sin exponerla en el repo.

Migraciones: `synchronize: true` solo mientras el modelo se estabiliza

> CORRECCIÓN El proyecto hoy corre con `synchronize: true` en `typeorm.config.ts` (marcado como TEMPORAL) para crear las 21 tablas sin escribir migraciones mientras el modelo todavía cambia. Antes de la entrega hay que pasar a `typeorm migration:generate`, poner `synchronize: false` y commitear las migraciones: lo que sigue explica por qué.

En solitario, dejar que TypeORM sincronice el esquema automáticamente es cómodo. En equipo es la fuente número uno de esquemas que divergen sin dejar rastro: cada uno modifica sus entidades por su lado y nadie sabe qué cambió el esquema real. Usen `typeorm migration:generate` desde el primer commit — las migraciones se revisan en el PR como cualquier otro código, y le dan a cada estudiante una forma confiable de reconstruir su base desde cero cuando algo se corrompe probando condiciones de carrera a propósito.

Seed de datos versionado

Un script de seed (roles base, un par de médicos, coberturas, camas de ejemplo) para que todos arranquen del mismo estado. Es lo que hace reproducibles las historias de triaje y concurrencia entre máquinas distintas — sin esto, "en mi compu funciona" va a aparecer seguido.

CI con PostgreSQL real, no mocks

Si usan GitHub Actions, corran PostgreSQL como *service container* con la misma imagen del compose (GitHub Actions trae soporte directo para `services: postgres:`, más simple de configurar que con SQL Server), apliquen las migraciones y corran ahí los tests de integración que fuerzan dos peticiones simultáneas contra el mismo turno o la misma cama. Es la única forma de demostrar que el locking optimista realmente funciona — un repositorio en memoria no reproduce una condición de carrera real.

DIVISIÓN DE TRABAJO SUGERIDA

Dado el plan de fases de la sección 09, el módulo de Admisión y Turnos (Fase 2) es el que más conviene resolver entre dos personas en pareja — concentra concurrencia, State y cola de prioridad a la vez. El resto de los módulos, una vez que ese patrón está probado, se pueden repartir uno por estudiante sin pisarse.

Informe de arranque basado en «Diseño de sistema» e «Historias de usuario» — próximo paso natural: levantar la Fase 0 y confirmar el modelo de datos de Admisión antes de tocar código de negocio.